PyTest-Based Software Testing for an AI Recruitment System

Problem Statement

Software systems must be thoroughly tested to ensure correctness, reliability, and maintainability. Manual testing is time-consuming and often fails to cover edge cases, boundary conditions, and negative scenarios. Automated testing frameworks such as **PyTest** help developers create comprehensive and reusable test suites for validating software behavior.

Develop a comprehensive **PyTest-based testing suite** for the given AI-powered Recruitment System. The test suite should verify the correctness of every component without modifying the provided source code and should include functional, boundary, edge-case, and negative testing.

** The required code is provided in resources.*

Objective

The objective of this assignment is to:

- Understand automated software testing using PyTest.
- Design unit tests for object-oriented software components.
- Apply functional, boundary, edge-case, and negative testing techniques.
- Use mocking where required for deterministic testing.
- Improve software reliability through comprehensive test coverage.

Features to Implement

- Write unit tests for the **ResumeParser** class.
- Write unit tests for the **MCQGenerator** class.
- Write unit tests for the **CandidateScorer** class.
- Write unit tests for the **InterviewEvaluator** class.

- Write unit tests for the **RecruitmentSystem** class.
- Test functional behavior of every component.
- Test boundary and edge cases.
- Test negative scenarios and exception handling.
- Use mocking where required (e.g., `random.choice`).
- Generate a comprehensive PyTest test suite without modifying the given source code.